DAILY JAVA DEEP DIVES · THE LATEST NEWS, FULLY EXPLAINED

Java – Beautifully Broken

Oracle Java gets up to 94% cheaper for small companies, and IntelliJ's brain moves into VS Code

Issue #7 | Sun, 11 Oct 2026 | ⏱️ ~30 min read | Level 3/5

☕ **Today's two stories**

1. **Oracle has cut the list price of a Java subscription for companies with fewer than 3,000 employees, in some cases by more than 90%.** A 100-person company now pays $1,000 a year instead of about $18,000. It comes nine days before Oracle JDK 21 stops getting free security updates for production use. Here's how the new prices work, who they apply to, and how they change the "pay Oracle or switch JDK" decision.
2. **JetBrains has put IntelliJ IDEA's Java engine into VS Code and Cursor.** Its new extension reached release candidate on 7 October, in the same fortnight that Oracle and Red Hat updated their own Java extensions for VS Code. All three run on the Language Server Protocol. Here's how that works, how the three differ, and why AI coding agents are part of the story.
3. **On the radar:** 6 more things that happened, listed at the end.

STORY 1 Oracle cuts the price of Java for small and mid-sized companies

LICENSING · ORACLE JAVA SE UNIVERSAL SUBSCRIPTION · ANNOUNCED 8 OCT 2026 · ~13 MIN READ

Imagine a 100-person company running a few Spring Boot services on Oracle JDK 21. Until this week, the published price for staying on Oracle's supported builds was about $18,000 a year. As of 8 October it's $1,000. Oracle hasn't changed what you get. It has changed how small companies are charged, at the very moment many of them have to decide whether to pay for Java at all. To see why the timing matters, and why the new price list has some odd edges, you need a short history of how Oracle charges for Java.

WHAT HAPPENED

On 8 October 2026, Donald Smith, Oracle's Vice President of Product Management for Java, announced "simplified pricing and purchasing of the Java SE Universal Subscription" for organisations with fewer than 3,000 employees.[1] The stated aim is to "make the benefits of the Java SE Universal Subscription more accessible to small and medium businesses."[1] The new annual prices, in US dollars, are:[1]

Organisation size	New annual price
10 employees or fewer	$100
100 employees or fewer	$1,000

Organisation size	New annual price
500 employees or fewer	$10,000
501 to 2,999 employees	$10,000, plus $147.20 for each employee above 500
3,000 employees or more	Unchanged (enterprise price list)

Organisations with 500 or fewer employees can now buy the subscription themselves through the Oracle Store, without talking to a salesperson. Larger organisations are asked to contact Oracle Sales.[1] Oracle's subscription FAQ has already been updated with the same tiers, describing the 501–2,999 band as "$10,000 per year, plus $147.20 annually for each employee above 500", and the enterprise price as starting at $10.50 per employee per month.[2]

The post doesn't say what the old prices were. That's where the size of the cut becomes clear.

THE BACKGROUND YOU NEED

Oracle JDK is one Java among many

When people say "Java" they usually mean a **JDK** ≈ *Java Development Kit: the compiler, the JVM that runs your code, and the standard library, packaged as one download*. Almost every JDK you can download is built from the same source code, **OpenJDK** ≈ *the open-source project, led by Oracle, where the Java platform is developed; its code is licensed under GPLv2 with the Classpath Exception*. Different vendors then build, test and ship it as their own **distribution** ≈ *a vendor's packaged build of OpenJDK, with its own licence, update schedule and support offer*: Eclipse Temurin, Amazon Corretto, Azul Zulu, Red Hat's builds, Microsoft's build, and Oracle's own.

Oracle actually ships two. Its OpenJDK builds on jdk.java.net use the open-source licence.[3] **Oracle JDK**, its commercial build, comes under Oracle's own licence terms, and those terms are what this story is about. If your servers run Temurin or Corretto, today's price change doesn't apply to you, though the rest of this story explains why many teams ended up on those distributions in the first place.

A decade of licence changes

For most of Java's history, Oracle's JDK was free for everyone under the **BCL** ≈ *Binary Code License: the licence Oracle JDK used until 2019, which allowed free commercial use*. That changed in January 2019, when Oracle stopped publishing free public updates of Java SE 8 for commercial users and introduced a paid subscription instead.[4] Later updates of Java 8, 11 and 17 came under the **OTN licence** ≈ *Oracle Technology Network License for Java SE: free for personal use, development, testing, prototyping and demos, but commercial production use needs a paid subscription*.[5,3]

Then, in September 2021, Oracle went the other way. It introduced the **NFTC** ≈ *Oracle No-Fee Terms and Conditions: a licence that allows free use of Oracle JDK, including commercial and production use, for a limited period*, which "permits free use for all users – even commercial and production use."[3] The catch is the time limit. Each **LTS** ≈ *Long-Term Support release: a Java version that vendors keep patching for years; currently Java 8, 11, 17, 21 and 25* release stays under the NFTC only until one year after the next LTS ships.[3] After that, updates move back to the OTN licence.

Updates matter because Oracle fixes security vulnerabilities through a scheduled **Critical Patch Update** ≈ *Oracle's quarterly security release (January, April, July, October) that bundles fixes across its products, including the JDK* (CPU). A JDK that stops getting CPUs slowly collects known, unpatched vulnerabilities.

For JDK 21, the clock is about to run out. Oracle's August blog post says "all updates through and including September 2026 are available under the NFTC", and that from the October 2026 CPU, JDK 21 updates will be offered under the OTN licence.[5] That CPU is due on 20 October. Oracle's suggested options are to migrate to JDK 25, which stays under the NFTC until October 2028, or to buy a subscription, which covers Java 21 until at least September 2031.[5]

How Oracle has licensed its JDK



Why the old price hurt small companies

In January 2023, Oracle replaced its older subscriptions with the **Java SE Universal Subscription**.[2] The big change was the unit of pricing. The old subscriptions were priced per processor (servers) or per named user (desktops).[6] The new one uses an **employee metric** ≈ *pricing based on your organisation's total headcount, not on how many people or machines use Java*. It counts everyone, not just developers: full-time and part-time workers, temporary employees, and the "contractors, outsourcers, agents, and consultants that support your internal business operations."[6]

The published price ladder started at $15 per employee per month for organisations with up to 999 employees, then fell with size: $12 up to 2,999, $10.50 up to 9,999, and so on down to $5.25 at 40,000–49,999.[7] So a 100-person company running one small Java service owed the same per-head price as one running hundreds: 100 × $15 × 12 = $18,000 a year.

That's why many teams treated the NFTC deadline as a migration deadline. Azul, which sells a competing JDK and support, warned in July that customers moving to the Universal Subscription could see licensing costs rise "by 2-5x", and described a "six- or seven-figure annual subscription" for enterprises that had never paid for Java before.[8] Azul is a competitor, so treat that as a vendor's view, but it shows how the 2023 model was seen.

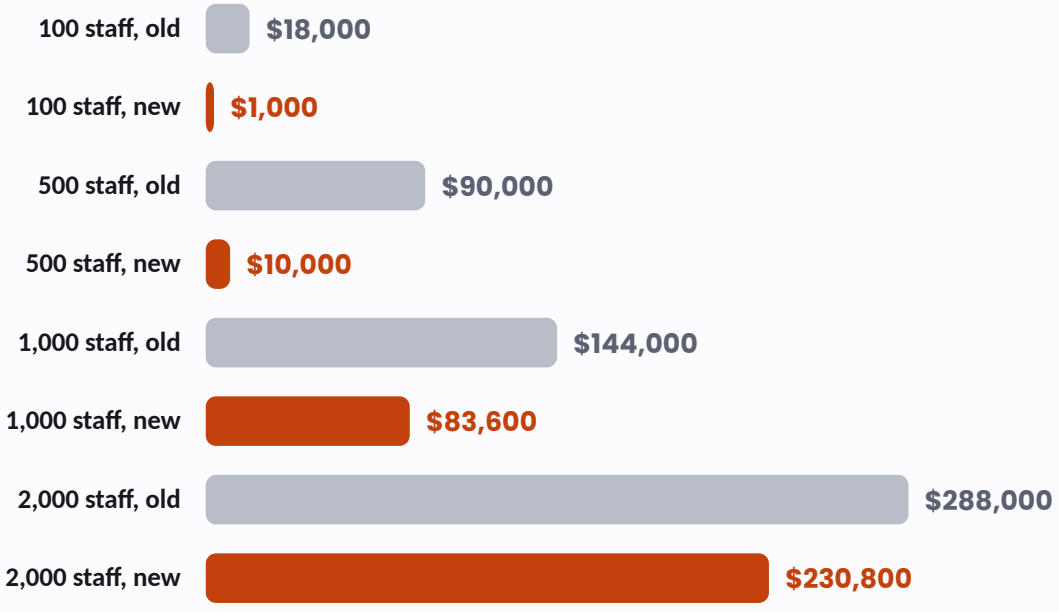
HOW IT WORKS

Bands instead of a per-head rate

Below 500 employees, the new model no longer multiplies headcount by a rate. It puts you in one of three **bands** and charges a flat price for the band. Between 501 and 2,999, it goes back to a per-employee charge on top of a $10,000 base. Here's what that does to the list price at a few sizes. The "old" column is our calculation from the published 2023 ladder;[7] the "new" column applies Oracle's 8 October tiers.[1]

Employees	Old list price per year	New price per year	Change
10	$1,800	$100	−94%
100	$18,000	$1,000	−94%
500	$90,000	$10,000	−89%
1,000	$144,000	$83,600	−42%
2,000	$288,000	$230,800	−20%
2,999	$431,856	$377,853	−13%
3,000	$378,000	$378,000	unchanged

Annual list price, old ladder vs new tiers (USD)



Our arithmetic. Old = headcount × $15 (under 1,000) or $12 (1,000–2,999) × 12 months, from the published ladder (Redress Compliance). New = Oracle's 8 Oct 2026 tiers. List prices; negotiated deals may differ.

The smaller you are, the bigger the cut.

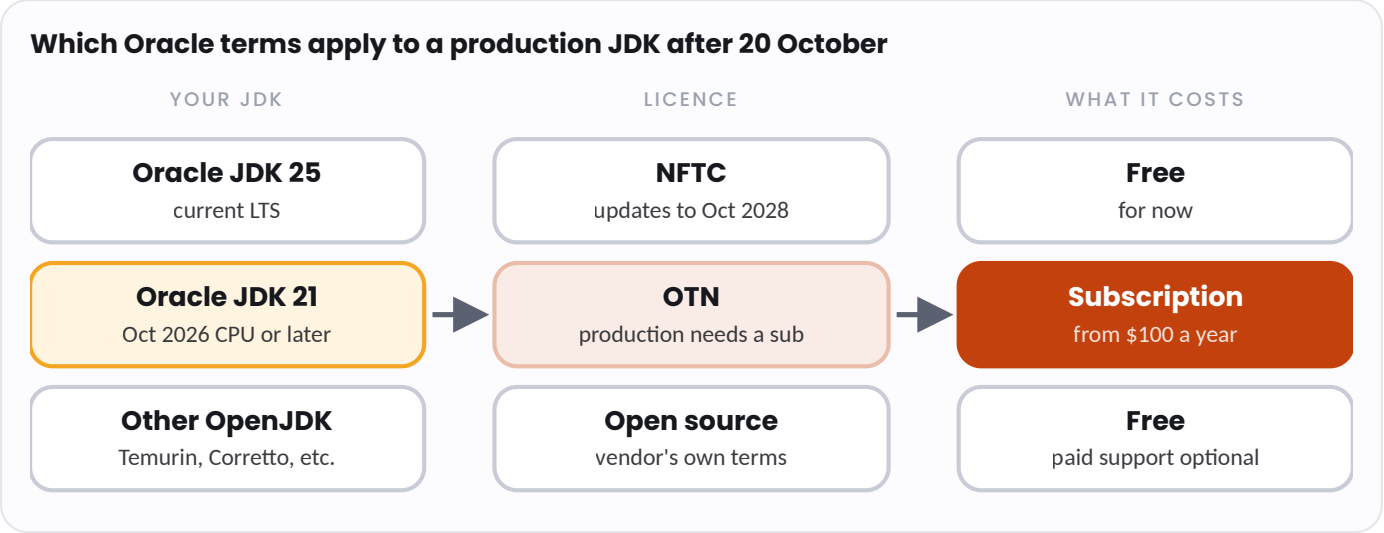
The odd number has a reason

Why $147.20 per extra employee, and not a round number? Do the sum at the top of the band. At 3,000 employees the enterprise rate applies: 3,000 × $10.50 × 12 = $378,000. Subtract the $10,000 base and divide by the 2,500 employees between 500 and 3,000, and you get exactly $147.20. In other words, the middle band is a straight line that runs from $10,000 at 500 employees to precisely the enterprise price at 3,000. Oracle doesn't explain the number, but the arithmetic shows it was chosen so there's no jump at the 3,000 boundary.

That fixes a strange feature of the old ladder. Because the rate fell from $12 to $10.50 at 3,000 employees, a company with 2,999 people paid about $54,000 a year *more* than one with 3,000. The same happened at 1,000: 999 employees cost $179,820, while 1,000 cost $144,000.

The new edges

Flat bands create their own edges. Ten employees pay $100, but eleven pay $1,000. A hundred pay $1,000, but 101 pay $10,000. Per employee per month, that's about 83 cents at exactly 10 or 100 people, and about $8.25 at 101. Even at those worst points it's still cheaper than the old $15, but if your headcount sits just above a boundary, it's worth knowing that contractors count too, and that the count is your organisation's, not your Java team's.



What the subscription includes

The price changed, but the product didn't. Oracle says the subscription gives "a commercial license for all versions of Oracle Java", access to Oracle's support and its performance, stability and security updates, and tools for managing and securing Java deployments.[1] The FAQ adds desktop, server and cloud use, My Oracle Support, and commercial support for components in the **Java Verified Portfolio**, Oracle's set of supported Java libraries and tools.[2] One example is Jipher, Oracle's FIPS-validated cryptography provider for regulated environments, whose new version 20 is included in that portfolio.[9] Oracle Java workloads running on Oracle Cloud Infrastructure get the subscription features at no extra cost.[1]

WHAT IT MEANS FOR YOU

First, find out whether you're affected at all. The question isn't "do we use Java?" but "do we run **Oracle JDK** in production?" You can see which build a machine uses from its properties:

```
$ java -XshowSettings:properties -version 2>&1 | grep -E 'runtime.name|java.vendor ='
    java.runtime.name = OpenJDK Runtime Environment
    java.vendor = Ubuntu
```

That's our sandbox, which runs Ubuntu's OpenJDK 21 build, so Oracle's commercial terms don't apply. Note that Oracle's open-source builds from jdk.java.net also name Oracle as the vendor, so where the JDK came from matters as much as the vendor string. Check container base images too, not only servers.

If you do run Oracle JDK 21 in production, you have four realistic paths before or soon after 20 October:

1. **Move to Oracle JDK 25.** It's the current LTS and stays free under the NFTC until October 2028.[5] You'd face the same decision again then, after Java 29 ships in September 2027.[3]
2. **Switch to another OpenJDK distribution.** Eclipse Temurin promises JDK 21 builds until "at least Dec 2029",[10] and Amazon Corretto 21 until October 2030, with Amazon stating it "does not charge for its use or distribution."[11] These are built from the same OpenJDK code, so for most server applications the switch is a base-image change plus a test run.
3. **Buy the subscription.** This is the option the announcement changes. For a company of 100, the choice is now between $1,000 a year and the engineering time to test a JDK swap across every service, base image and build pipeline. For many small teams, that time costs more than $1,000. The subscription also brings Oracle support, which free distributions don't include unless you buy it from their vendor.
4. **Stay on 21.0.12.1, the last NFTC build Oracle has published, and stop patching.** This is the option to avoid. Every quarter without updates means more publicly known, unpatched vulnerabilities.

There's also a less obvious group: companies that never noticed they were using Oracle JDK in production. Licensing consultancies say Oracle stepped up Java compliance checks after the 2023 change and that the employee metric made under-licensing expensive. One describes Oracle reps' "routine account review" calls as "discovery exercises."[12] Those are claims by firms that sell audit defence, not Oracle statements, but the practical point holds: a $100 or $1,000 subscription is a cheap way for a small company to make that risk go away.

For mid-sized companies, between 1,000 and 3,000 employees, the cut is real but smaller: 20–42% off list at the sizes in our table. The free distributions remain a much bigger saving, so the decision there still depends on whether you value Oracle's support enough to pay for it.

💥 Watch out for

- **Headcount means the whole organisation.** The employee metric counts part-timers, temporary staff and contractors who support your business, not only people who use Java.[6]
- **Band edges.** Eleven employees cost ten times as much as ten; 101 cost ten times as much as 100.[1]
- **These are list prices.** Oracle's post doesn't mention contract length, minimum terms or discounts, and online purchase is only for organisations of 500 or fewer.[1]
- **Development use is still free under OTN, but read it carefully.** Oracle's FAQ says the OTN licence covers "personal use, development, testing, prototyping, demonstrating", but also that using Java on a work laptop "as part of any business operations" is not personal use.[3]
- **Nothing changes at 3,000+.** Large enterprises keep the existing per-employee ladder.[1]

WHAT HAPPENS NEXT

The October CPU on 20 October brings the first JDK 21 update under the OTN licence.[5] JDK 25 stays under the NFTC until October 2028, a year after Java 29 LTS.[5] Oracle hasn't said whether this pricing is permanent or how it will treat existing contracts at renewal. If you have a renewal coming up and fewer than 3,000 employees, it's worth asking.

📖 Words from this story

JDK: Java Development Kit, the compiler, JVM and standard library packaged as one download.

OpenJDK: the open-source project where Java is developed, licensed under GPLv2 with the Classpath Exception.

Distribution: a vendor's packaged build of OpenJDK, such as Temurin, Corretto or Oracle JDK.

BCL: Binary Code License, Oracle JDK's licence until 2019, which allowed free commercial use.

OTN licence: Oracle Technology Network License for Java SE; free for personal use, development and testing, while commercial production use needs a subscription.

NFTC: Oracle No-Fee Terms and Conditions, free use including production, until one year after the next LTS.

LTS: Long-Term Support release, a Java version patched for years (8, 11, 17, 21, 25).

Critical Patch Update (CPU): Oracle's quarterly security release in January, April, July and October.

Employee metric: pricing based on total organisational headcount, including contractors, regardless of who uses Java.

STORY 2 IntelliJ's Java engine moves into VS Code and Cursor

TOOLING · JETBRAINS, ORACLE, RED HAT · RC RELEASED 7 OCT 2026 · ~13 MIN READ

For as long as most Java developers can remember, if you wanted IntelliJ IDEA's Java refactorings and inspections, you had to use IntelliJ IDEA. That's changing. On 7 October, JetBrains released the final test build of an extension that runs IntelliJ's Java and Kotlin engine inside VS Code and Cursor. In the same fortnight, Oracle and Red Hat shipped new versions of their own Java extensions for VS Code. The editor you type in and the engine that understands your code are separating, and AI coding agents are one of the reasons JetBrains says this matters.

WHAT HAPPENED

JetBrains' extension is called **Java and Kotlin by IntelliJ IDEA**. It reached release candidate on 7 October 2026.[13] The announcement, by Andrei Kogun, says the RC "is part of the Early Access Program and can be used for 30 days without a license", that the stable release "will arrive very soon", and that afterwards "new users will receive a 30-day free trial", after which the extension requires an IntelliJ IDEA Ultimate subscription.[13] JetBrains hasn't given a GA date.

The extension was first released as a preview two months earlier. In that announcement, Marco Behler wrote that "IntelliJ IDEA's Java and Kotlin intelligence is available in a preview extension for third-party editors", naming "VS Code and its forks (such as Cursor)".[14] Since then, JetBrains says, it has been downloaded more than 20,000 times from the Visual Studio Marketplace and the Open VSX Registry, and the team has tested it on the very large repository used to build IntelliJ-based IDEs themselves.[13] The RC offers completion, navigation, inspections and quick-fixes, project-wide refactorings, debugging, Maven, Gradle and Bazel projects, built-in test running, and "Spring tooling interoperability."[13]

The other two Java extensions for VS Code also had releases:

- **Red Hat's Language Support for Java**, version 1.57.0, came out on 1 October. It adds Java 27 support, faster startup, and a bundled JDK upgraded to 25.[16] It now needs Java 25 or later to run its language server.[17]
- **Oracle's Java Platform Extension**, version 27.0.0, was merged on 6 October and announced on Inside Java on 8 October.[18,19] It moves to NetBeans 31 for "improved support and tooling for JDK 27", adds breakpoints inside individual lambdas, and now needs JDK 21 or newer to run, while still working with projects on JDK 8 and above.[20]

THE BACKGROUND YOU NEED

What your IDE actually knows about your code

When you press "Go to definition" in a Spring project, your editor has to answer a hard question: which of the thousands of classes on the **classpath** ≈ *the list of directories and JAR files where Java looks for classes when compiling and running* does this name refer to? To answer it, an IDE does most of what a compiler does. It reads your

`pom.xml` or `build.gradle` to work out the dependencies, parses every file into an **abstract syntax tree** *≈ a tree-shaped representation of source code, in which each node is a construct such as a class, method call or variable*, resolves every name to a type, and keeps all of that up to date as you type. Completion, error highlighting, "find usages", refactoring and inspections are all built on top of that model.

Building that model well is years of work, and for a long time each IDE had its own. Eclipse has the **Eclipse JDT** (Java Development Tools). Oracle's NetBeans-based tooling builds on `javac`, the JDK's own compiler. IntelliJ IDEA has JetBrains' own engine, which is the thing many Java developers pay for.

The Language Server Protocol

The problem with every editor having its own engine is multiplication. If there are *m* editors and *n* languages, someone has to write *m* × *n* integrations. Microsoft's answer was the **Language Server Protocol** *≈ LSP: a standard way for an editor to ask a separate program questions about code, such as "where is this defined?" or "what are the errors in this file?"*. As the community site langserver.org puts it, LSP "creates the opportunity to reduce the *m-times-n* complexity problem": each language builds one server, each editor builds one client.[22] The protocol's public GitHub repository was created by Microsoft in 2015.[23]

A **language server** *≈ a background program that holds the model of your code and answers the editor's LSP requests* "runs as a separate process and development tools communicate with the server using the language protocol over JSON-RPC," according to the official overview.[24] **JSON-RPC** *≈ a simple convention for calling functions on another process by sending JSON messages that name a method and its parameters* keeps the messages simple. When you open a file, the editor sends `textDocument/didOpen`. As you type, it sends `textDocument/didChange`, and the server replies with `textDocument/publishDiagnostics` (your red squiggles). "Go to definition" is a `textDocument/definition` request.[24] This is the request from the LSP overview, with the file path shortened:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "textDocument/definition",
  "params": {
    "textDocument": { "uri": "file:///.../use.cpp" },
    "position": { "line": 3, "character": 12 }
  }
}
```

The server answers with another file URI and a line-and-character range. The point is what the protocol *doesn't* contain: no syntax trees, no types, no compiler symbols. The overview explains that standardising "a text document URI or a cursor position" is much simpler than standardising "an abstract syntax tree and compiler symbols across different programming languages."[24] All the language knowledge stays inside the server. That's what makes it possible to move a whole IDE engine behind the protocol.

Debugging has a similar standard, the **Debug Adapter Protocol** *≈ DAP: a standard protocol editors use to talk to debuggers, the debugging counterpart of LSP*, which JetBrains' extension uses for its debugger.[14]

How Java got three language servers

Red Hat was first. The **Eclipse JDT Language Server** *≈ JDT LS: an open-source Java language server built from the Eclipse IDE's Java tooling, used by Red Hat's VS Code extension* was committed to the Eclipse Foundation in February 2017.[25] Its README says it's built from Eclipse JDT for Java support, M2Eclipse for Maven and Buildship for Gradle.[25] Red Hat's VS Code extension runs it.

Oracle followed in October 2023 with its Java Platform Extension, which builds, runs and debugs Java in VS Code.[26] Its selling point is the compiler underneath: Oracle says it "is based on the OpenJDK javac compiler for code editing and compilation."[27] Because the editor uses the same compiler as the JDK, it can understand new language features as soon as `javac` does. By January 2025 it had passed 2 million downloads.[27] The extension's source shows it's built from the Apache NetBeans language server.[21]

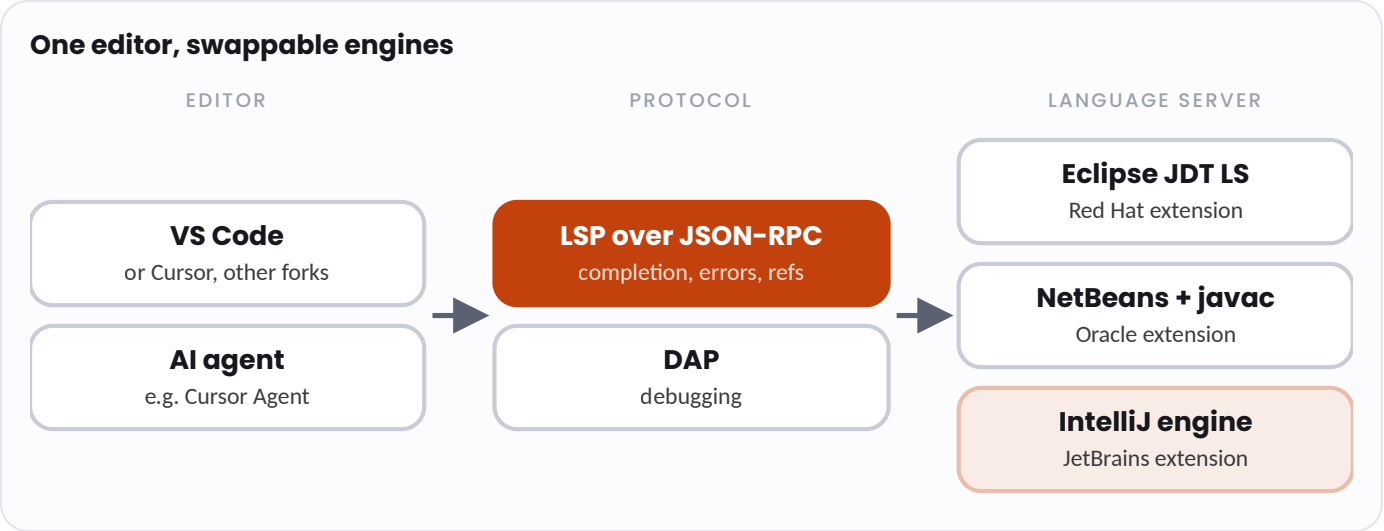
JetBrains held out longest. Its IDE business depends on people choosing IntelliJ, and DevClass noted in 2025 that the company considered its own language API better than LSP and faced a dilemma: an LSP server might reduce demand for its IDEs, while not having one could hurt Kotlin in a world where VS Code is popular.[28] Back in 2011, JetBrains had said it expected Kotlin "to drive the sales of IntelliJ IDEA."[28] Its first step was a pre-alpha Kotlin language server in May 2025, meant for "toy projects."[28] This month's RC covers Java as well, and it's built for real projects.

Java's road to three language servers



HOW IT WORKS

All three extensions follow the same pattern. A thin piece of code inside VS Code (the client) starts a language server process on your machine, using a JDK you provide or one it bundles. The server imports your build, which is why each extension needs to understand Maven and Gradle, builds its model of the code, and answers the editor's requests. What differs is the engine inside the server.



Here's how the three compare, using what each project says about itself:

	Red Hat 1.57.0	Oracle 27.0.0	JetBrains RC
Engine	Eclipse JDT, M2Eclipse, Buildship[25]	Apache NetBeans, `javac`[27]	IntelliJ IDEA[14]
JDK to run the server	25+ (bundled on most platforms)[17]	21+[20]	Not documented
Project JDKs	1.8 and above[17]	8 and above[20]	Java 25, 27 EA listed[15]
Build tools	Maven, Gradle[25]	Maven, Gradle[21]	Maven, Gradle, Bazel[13]
Price	Free, open source	Free, Apache 2.0	Ultimate subscription after trial[13]

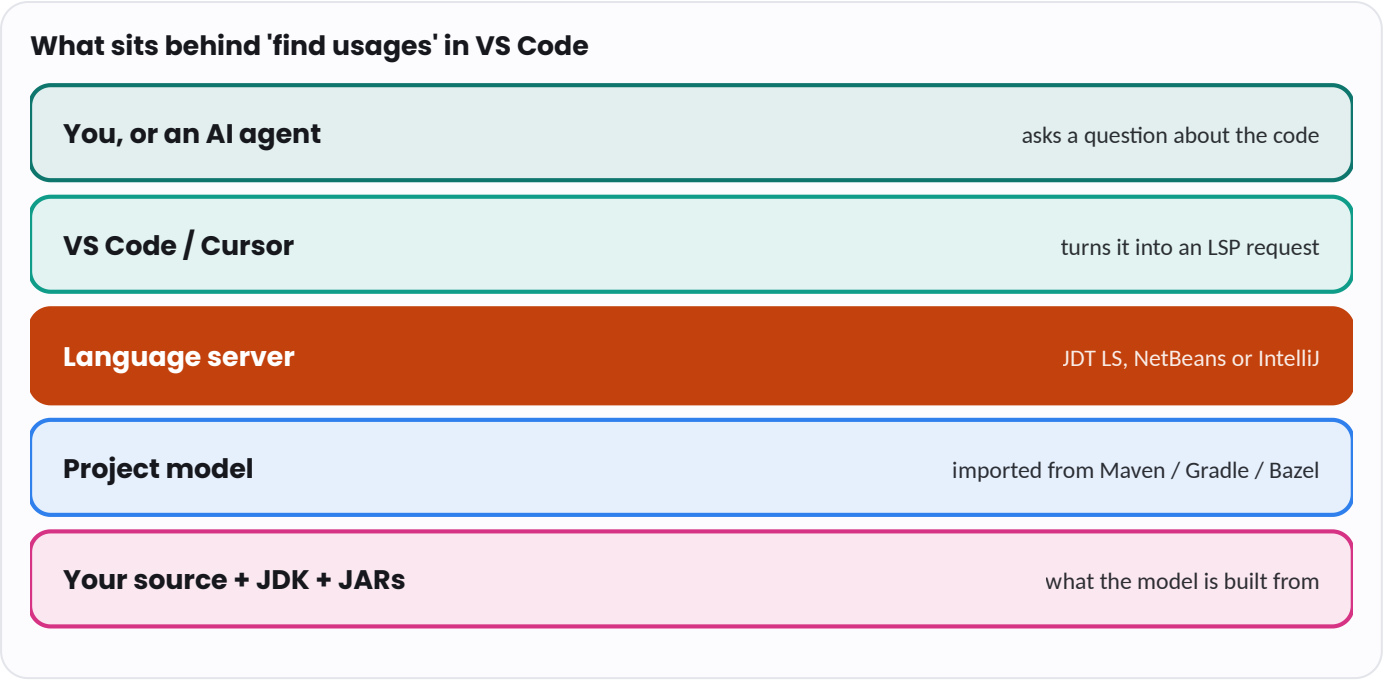
One practical detail: the three overlap. JetBrains' preview post says its extension's code analysis and quick-fixes overlap with Red Hat's and Oracle's extensions, and recommends disabling those while you use it.[14] Running two Java language servers on the same project means two models of your code in memory and duplicate warnings.

Why AI agents care

The new part of this story is who else is asking questions. An AI coding agent working in your repository needs the same answers an IDE does: where is this method called, what type is this variable, will this rename break anything? Without a language server, an agent mostly searches text. That's how you get a rename that misses an overload, or "usages" that are really a method with the same name in another class.

JetBrains says that when its extension is installed in Cursor, "Cursor Agent can use the same language server for semantic code search, navigation, and code analysis, leading to faster and more deterministic results."[13] The August post went further, saying internal trials suggest the same LSP functionality can significantly improve terminal-based agents such as Claude Code or Codex, with results to be published "soon."[14] Those numbers haven't been published yet, so treat this as a direction rather than a measured result. But it explains

the timing: a language server that answers precise questions about code is useful to people and to agents alike.



WHAT IT MEANS FOR YOU

If you already write Java in VS Code or Cursor, you now have three serious engines to choose from. Red Hat's is free, open source and updated roughly monthly.[16] Oracle's is free, tracks `javac` closely, and is a good fit if you want new language features early. JetBrains' brings IntelliJ's inspections and refactorings, but costs an Ultimate subscription once it's stable. You can try the RC now for 30 days: it's listed as `JetBrains.intellij-server` on the Visual Studio Marketplace and `JetBrains/intellij-server` on Open VSX.[13] Disable the other Java extensions first.

If your company already pays for IntelliJ IDEA Ultimate, this is mostly good news. JetBrains says the same licence covers IntelliJ's Java and Kotlin features in the desktop IDE, VS Code-based editors and other supported environments.[14] A developer who prefers Cursor for AI work no longer has to give up IntelliJ's analysis to use it, and a team that mixes editors can get the same inspections across all of them.

If you check upgrade prerequisites, note the JDK requirements. Oracle's extension needs JDK 21 or newer to run its server and Red Hat's needs Java 25, although Red Hat bundles a runtime in its platform-specific builds.[20,17] Your *project* can still target Java 8: the server's JDK and your project's JDK are separate settings in both extensions.

If you're evaluating AI coding tools, ask whether the agent uses a real language server or text search. With Java, the difference shows up in refactorings across a large codebase.

💥 Watch out for

- **It's still a release candidate.** The RC works for 30 days without a licence, and there's no GA date beyond "very soon."[13]
- **It's not free after the trial.** Unlike Red Hat's and Oracle's extensions, and unlike JetBrains' Apache 2.0 Kotlin-only language server, it needs IntelliJ IDEA Ultimate.[14]
- **One Java server at a time.** JetBrains recommends disabling the Red Hat and Oracle extensions to avoid overlapping analysis.[14]
- **Agent gains aren't measured yet.** JetBrains' terminal-agent results haven't been published.[14]
- **Resource use isn't documented.** JetBrains' documentation doesn't say how much memory the background server needs, or which JDK it runs on.[15]

WHAT HAPPENS NEXT

JetBrains says the stable release is close, and that future posts will cover "fully agentic, terminal-based workflows."[13] Red Hat's extension has been shipping roughly monthly, and Oracle's version numbers track JDK releases, so both will need updates when JDK 28 arrives in March 2027. The bigger question is commercial: will a good-enough free language server keep developers in VS Code, or will IntelliJ's engine in VS Code pull more of them into paying for Ultimate? We'll know more once JetBrains publishes its agent results and its stable release.

📖 Words from this story

Classpath: the directories and JAR files where Java looks for classes when compiling and running.

Abstract syntax tree: a tree representation of source code, with a node for each construct.

Language Server Protocol (LSP): a standard way for an editor to ask a separate program questions about code.

Language server: a background program that holds the model of your code and answers LSP requests.

JSON-RPC: calling functions on another process by sending JSON messages that name a method and its parameters.

Debug Adapter Protocol (DAP): a standard protocol editors use to talk to debuggers.

Eclipse JDT Language Server (JDT LS): the open-source Java language server built from Eclipse's Java tooling, used by Red Hat's extension.

On the radar

Also happened, in one line each. Some will get a full deep dive in a future issue.

- **Oracle Jipher 20:** Oracle's FIPS cryptography provider for Java adds ML-KEM and ML-DSA post-quantum algorithms and moves its native layer to the FFM API on JDK 25+.[9]
- **Hibernate ORM 8.0.0.Beta3:** the third beta of the next Hibernate major is on Maven Central.[29]
- **Jackson patch wave:** jackson-databind 3.2.3, 3.1.7, 2.22.3, 2.21.7 and 2.18.11 shipped on 20–22 September.[30]
- **Gradle 9.9.0-M2:** the second milestone of Gradle 9.9 arrived on 18 September, a week before 9.8.0 went stable.[31]
- **Kohsuke Kawaguchi on A Bootiful Podcast:** the creator of Jenkins is this week's guest on Spring's podcast.[32]
- **Java Annotated Monthly, October 2026:** JetBrains' monthly round-up covers Java 27, JDK 28 previews and Spring.[33]

---

1. **New Lower SMB Pricing and Easier Online Purchasing of Java SE Universal Subscription**, Donald Smith, Oracle Java blog, 8 Oct 2026.
2. **Java SE Universal Subscription FAQ**, Oracle, accessed 11 Oct 2026.
3. **Oracle Java SE Licensing FAQ**, Oracle, accessed 11 Oct 2026.
4. **Oracle Java Licensing Changes Timeline**, Oracle Licensing Experts.
5. **JDK 21 approaches end-of-permissive license**, Donald Smith, Oracle Java blog, 14 Aug 2026.
6. **Java SE Universal Subscription: Oracle's New Licensing Model**, UpperEdge, 15 Feb 2023.
7. **Oracle Java Licensing Cost 2026: Updated Pricing**, Redress Compliance, updated 23 Sep 2026.
8. **The Java 21 Doomsday Clock Countdown**, Dana Crane, Azul, 29 Jul 2026.
9. **Announcing Oracle Jipher 20: FIPS 140-3 Cryptography for Java**, Poonam Parhar, Oracle Java blog, 15 Sep 2026.
10. **Temurin Support**, Eclipse Adoptium, accessed 11 Oct 2026.
11. **Amazon Corretto FAQs**, Amazon Web Services, accessed 11 Oct 2026.
12. **Why Oracle Java Audits Have Exploded Since 2023**, Redress Compliance (an audit-defence consultancy).
13. **Java and Kotlin by IntelliJ IDEA: The LSP-Based Extension Reaches Release Candidate**, Andrei Kogun, JetBrains, 7 Oct 2026.
14. **IntelliJ IDEA Goes LSP: Java and Kotlin Intelligence Comes to VS Code, Cursor, and Agentic Flows**, Marco Behler, JetBrains, Aug 2026.
15. **About Java and Kotlin by IntelliJ IDEA**, JetBrains documentation, 28 Jul 2026.
16. **vscode-java CHANGELOG, 1.57.0 (1 Oct 2026)**, Red Hat, GitHub.
17. **Language Support for Java by Red Hat, README**, Red Hat, GitHub.
18. **Merge pull request #657 from oracle/27.0.0**, oracle/javavscode, GitHub, 6 Oct 2026.
19. **Inside.java front page: "Oracle Java Platform Extension for Visual Studio Code - Version 27.0.0 Is Now Available"**, Inside Java, 8 Oct 2026.
20. **javavscode CHANGELOG, Version 27.0.0**, Oracle, GitHub.
21. **oracle/javavscode README and repository**, Oracle, GitHub.
22. **Langserver.org**, Sourcegraph.
23. **microsoft/language-server-protocol**, Microsoft, GitHub (initial commit 4 Sep 2015).
24. **Language Server Protocol: Overview**, Microsoft.
25. **Eclipse JDT Language Server, README**, Eclipse Foundation, GitHub (initial commit 9 Feb 2017).
26. **Oracle Introduces Java Platform Extension for Visual Studio Code**, Johan Janssen, InfoQ, 19 Dec 2023.
27. **Oracle Java Platform VS Code Extension Reaches 2M Downloads**, Erin Dawson, Oracle, 15 Jan 2025.
28. **JetBrains previews official VS Code language server for Kotlin**, Tim Anderson, DevClass, 23 May 2025.

29. **hibernate-core 8.0.0.Beta3**, Maven Central.

30. **FasterXML/jackson-databind releases**, ReleaseAlert.

31. **gradle/gradle-distributions releases**, Gradle, GitHub.

32. **A Bootiful Podcast: legendary Java developer Kohsuke Kawaguchi**, Spring, 8 Oct 2026.

33. **Java Annotated Monthly, October 2026**, JetBrains.